



Form Handling In React

useForm() hook come with multiple utilities that we can use for below purposes.

- a) **register:** - to bind form fields to the input , it accepts two parameter argument.
 - 1. Form field name
 - 2. 2nd optional :this optional argument may accepts the validation criteria to the user input in form of object.
- b) **handleSubmit:** - this property contains a function expression which we us to handle submit event of form. The function expression accepts a callback function as parameter argument to give form data.
- c) **reset:** - this property also contains a function expression which is use to reset the value of all form fields.
- d) **setValue:** - the setValue property of useForm() hook contains a function expression, and it is use to assign the value to the desired form field by program / after executing some logic.

The setValue function expression accepts two parameter argument.

- 1. Field name
 - 2. Value which is need to assign
- e) **formState:{errors}:** - the formState is a nested property which maintains the array of errors which are raised by wrong user inputs passed to validated form filed.



Q) How to declare the validation of form field in object format?

-> We have to use below properties to apply validation in validation object.

1. **required:** - indicated that the form field is required to set the value.
2. **minLength:** - set the limit of minimum character that we have enter in input.
3. **maxLength:** - set the maximum limit of characters that we have to enter in input.
4. **min:** - the min property is use to set the limit of smallest number that we can store in form field.
5. **max:** - the max property is use to set the limit of largest number that we can store in form field.
6. **pattern:** - the pattern property validates the user input according to the regular expression assigned to it.

Note: - All the above mentioned properties of validation object contains the reference of another object (nested object)

The nested object has below properties: -

- a) **value:** - to specify the value of validation
- b) **message:** - to specify the default message, which is needed to render on browser if the value entered by ender user, not satisfy by validation criateria.



CJC

Complete Java Classes

by Kunal Sir

JS Register.js U X

src > component > register > JS Register.js > Register > fieldValidations > firstName > required > message

```
1  import { useForm } from 'react-hook-form';
2  import './Register.css'
3
4  function Register()
5  {
6
7      const {register ,handleSubmit ,reset ,setValue, formState:{errors}} = useForm()
8      const fieldValidations={
9          firstName:{ //Pratik
10             required:{value:true ,message:'Firstname is must..!'},
11             minLength:{value:3,message:'Firstname must have atleast three later..!'},
12             maxLength:{value:12,message:'Firstname must contain only 12 latters..!'},
13             pattern:{value:/[A-Z][a-z]+@[0-9]/ , message:'First later must be in upper case'}
14         },
15         lastName: {
16             required:{value:true ,message:'LastName is must..!'},
17             minLength:{value:3,message:'Lastname must have atleast three later..!'},
18             maxLength:{value:12,message:'Lastname must contain only 12 latters..!'}
19         },
20         age:{
21             required:{value:true,message:'Age is required..!'},
22             min:{value:21,message:'user must have minimum age 21 years'},
23             max:{value:80,message:'user age may not be greater than 80 year'},
24             pattern:{ value:/[0-9]/ , message:'Age must be a number'}
25         }
26     }
```

```
27     function getFromData(fromData)
28     {
29         console.log(fromData)
30
31         reset();
32     }
33     const onSetValues=()=>{
34         alert('onSetValue called')
35         setValue('firstName','Sham');
36         setValue('lastName','Patil')
37     }
38     return (<div className='box'>
39         <h1> Sign-Up Here..!</h1>
40
41         <form onSubmit={handleSubmit(getFromData)}>
42             <p className='message'>{errors.firstName && errors.firstName.message}</p>
43             <label>Enter First Name</label>
44             <input type='text' {...register('firstName',fieldValidations.firstName)}><br/><br/>
45
46             <p className='message'>{errors.lastName && errors.lastName.message}</p>
47             <label>Enter Last Name</label>
48             <input type='text' {...register('lastName',fieldValidations.lastName)}><br/><br/>
49             <p className='message'>{errors.age && errors.age.message}</p>
50             <label>Enter Age </label>
51             <input type='text' {...register('age',fieldValidations.age)}><br/><br/>
```



CJC

Complete Java Classes

by Kunal Sir

```
52
53         <label>Enter Email id</label>
54         <input type='text' {...register('email')}/><br/><br/>
55
56         <button type='submit'>Submit</button>
57         <button type='button' onClick={()=>reset()} >Reset</button>
58         <button type='button' onClick={onSetValues}>Set Value</button>
59     </form>
60
61 </div>
62 )
63
64 }
65
66 export default Register;
```



CJC

Complete Java Classes